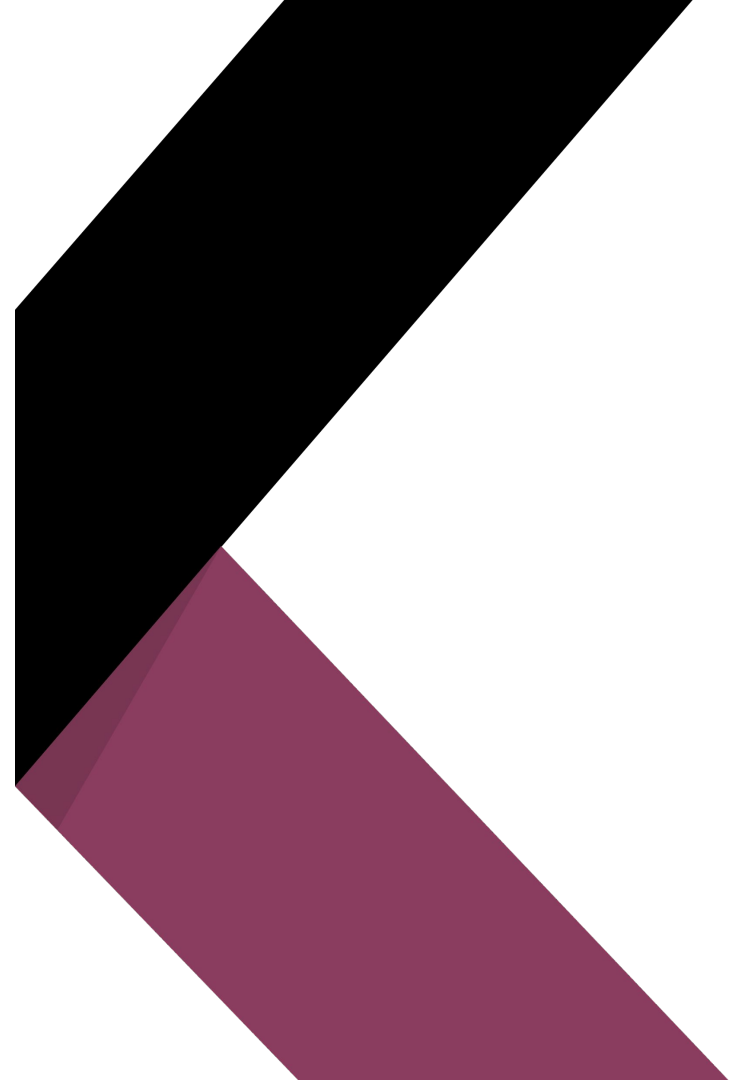


CAN - Analysis

CANalyzer





CAN Network



CAN - Introduction

The CAN (Controller Area Network) bus is broadcast digital bus designed to operate at speeds from 20kb/s to 1Mb/s. The transmission rate depends on the bus length and transceiver speed.

The CAN network protocol has been defined to provide deterministic communication in complex distributed systems with the following features/capabilities:

- Possibility of assigning priority to messages and guaranteed maximum latency times.
- Multicast communication with bit-oriented synchronization.
- System wide data consistency.
- Multimaster access to the bus.
- Error detection and signalling with automatic retransmission of corrupted messages.
- Detection of possible permanent failures of nodes and automatic switching off of defective nodes.



CAN - Network

- The interface between a CAN controller chip and a two-wire differential bus typically consists of a transmitting and a receiving amplifier (transceiver = transmit and receive).
- The transceiver must convert the electrical representation of a bit from the one in use by the controller to the one defined for the bus.
- In addition, it must provide sufficient output current to drive the bus electrical state, and protect the controller chip against overloading.
- As a receiver, the CAN transceiver provides the recessive signal level and protects the controller chip input comparator against excessive voltages on the bus lines.
- Furthermore, it detects bus errors such as line breakage, short circuits, shorts to ground, etc.
- A further function of the transceiver can also be the galvanic isolation between a CAN node and the bus line.



CAN - Transmission rate & Bus lenght

- Because of the definition of the bit time, it clearly exists a dependency between the bit time and the signal propagation delay, that is, between the maximum achievable bit rate (or transmission speed) and the length of the bus
- The signal propagation delay to be considered for the computation of the maximum allowed bus length includes several stages
- For short, high speed networks, the biggest limitation to bus length is the transceivers propagation delay.

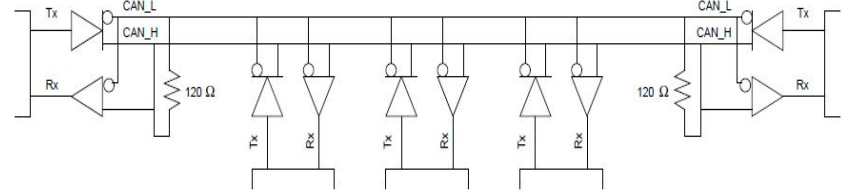
Bit rate	bit time	bus length
1 Mb/s	1 μ s	25m
800 kb/s	1.25 μ s	50m
500 kb/s	2 μ s	100m
250 kb/s	4 μ s	250m
125 kb/s	8 μ s	500m
62.5 kb/s	16 μ s	1000m
20 kb/s	50 μ s	2500m
10 kb/s	100 μ s	5000m



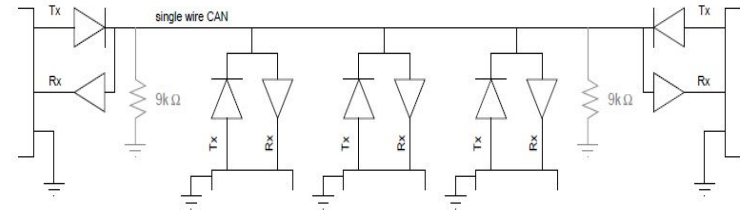
CAN - Bus termination

Electrical signals on the bus are reflected at the ends of the electrical line unless measures are taken. For the node to read the bus level correctly it is important that signal reflections are avoided. This is done by terminating the bus line with a termination resistor at both ends of the bus and by avoiding unnecessarily long stubs lines of the bus

- For high speed CAN, ISO 11898 requires a cable with a nominal impedance of 120 ohms, and therefore 120 ohm resistors should be used for termination (only the devices on the ends of the cable need termination resistors).
- For Low-Speed CAN, each device on the network needs a termination resistor for each data line.



Terminating a High-Speed Network.

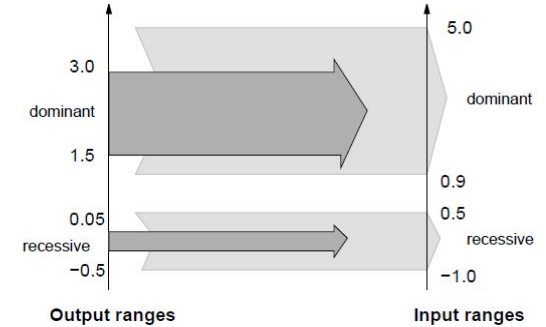
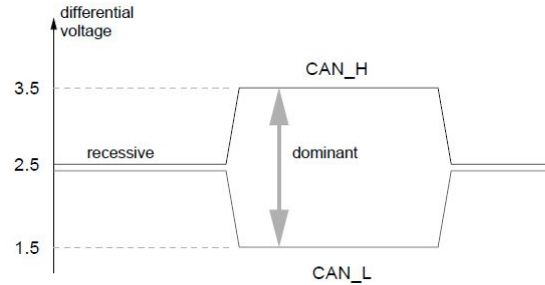


Termination Resistors on a Low-Speed Network.



CAN - Physical encoding of Dominant and recessive states

- CAN specifies two logical states: recessive and dominant. According to ISO-11898-2, a differential voltage is used to represent recessive and dominant states.
- In the recessive state (usually logic 1), the differential voltage on CAN H and CAN L is less than the minimum threshold ($<0.5V$ receiver input or $<1.5V$ transmitter output). In the dominant state (logic 0), the differential voltage on CAN H and CAN L is greater than the minimum threshold.



Encoding of dominant and recessive bits.



CAN - Message Format

In CAN, there are 4 different types of frames, according to their content and function:

- **DATA FRAMES** contain data information from a source to possibly multiple receivers.
- **REMOTE FRAMES** are used to request transmission of a corresponding (with the same identifier) DATA FRAME.
- **ERROR FRAMES** are transmitted whenever a node on the network detects an error.
- **OVERLOAD FRAMES** are used for flow control, to request an additional time delay before the transmission of a DATA or REMOTE frame.



CAN - Data frame

DATA frames are used to transmit information between a source node and one of more receivers. CAN frames do not use explicit addressing for identifying the message receivers, but each node defines the messages that will be received based on their information content, which is encoded in the IDENTIFIER field of the frame.

The structure of the DATA frames is:

- Identifier field;
- Control field;
- Data field;
- CRC and acknowledgment field;
- Interframe space.

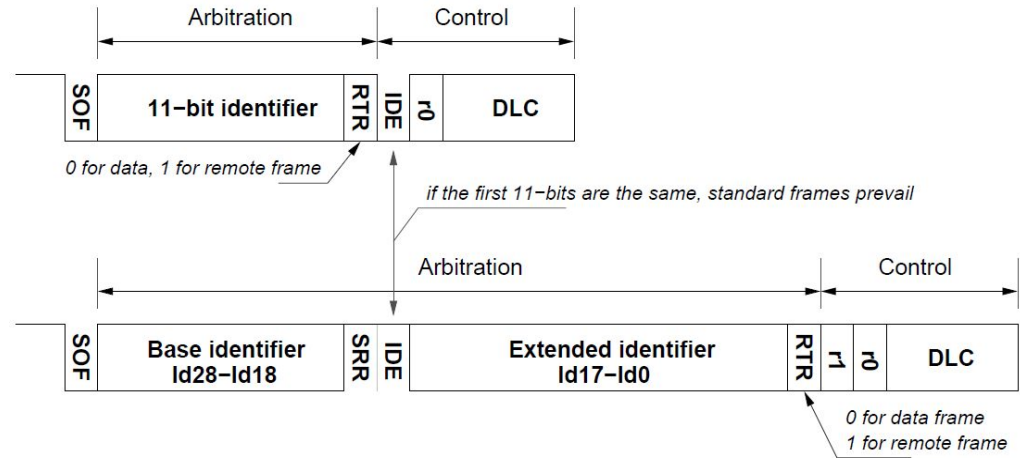


CAN - Data frame (Identifier)

The CAN protocol requires that all contending messages have a unique identifier. The identifier field consists of 11 (+1) bits in **standard** format and 29 (+3) bits in **extended** format.

The field is built:

- the first 11 bits of the identifier;
- RTR (Remote Transmission Request) bit in the standard and extended format;
- SRR (Substitute Remote Request), only in extended format; it's only a place holder
- Single IDE bit (Identifier Extension, always recessive)
- the remaining 18 least significant identifier bits





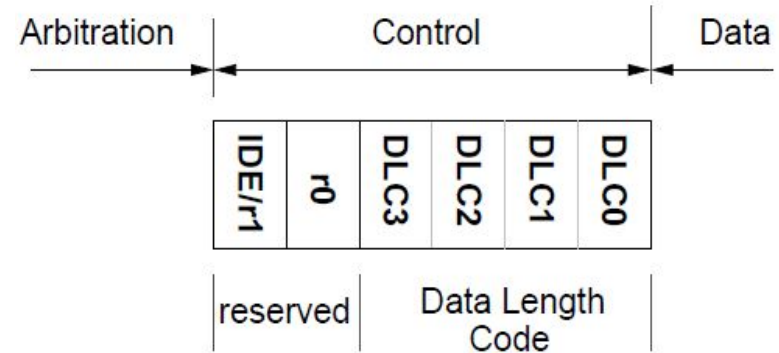
CAN - Data frame (Control, Data and Interframe space)

The **control field** contains 6 bits. The first two bits are reserved or predefined in content.

The following four bits define the length of the data content (Data Length Content, or DLC) in bytes. If the dominant bit is interpreted as 1 (contrary to the common notation in which it is read as 0) and the recessive as 0, the four DLC bits are the unsigned binary coding of the length.

The **data field** position is on the right side of the control field. The maximum dimension of the data field is 8 bytes.

Data frames and remote frames are separated by an **INTERFRAME** space (7 recessive bits) on the bus.





CAN - Remote/Error/Overload frame

A **remote** frame is used to request the transmission of a message with a given identifier from a remote node. A remote frame has the same format of a data frame with the following characteristics:

- the identifier field is used to indicate the identifier of the requested message
- the data field is always empty (0 bytes)
- the DLC field indicates the data length of the requested message (not the transmitted one)
- the RTR bit in the arbitration field is always set to recessive

The **error** frame is not a true frame, but rather the result of an error signalling and recovery. The details of such an event are described in the following sections.

The ERROR FRAME and the **OVERLOAD** FRAME are of fixed form as well and not coded by the method of bit stuffing



CAN - Error management

The CAN protocol is designed for safe data transfers and provides mechanisms for **error detection**, signalling and self-diagnostics, including measures for fault confinement, which prevent faulty nodes from affecting the status of the network.

The general measures for error detection are based on the capability of each node of monitoring broadcast transmissions over the bus, whether it is a transmitter or a receiver, and to signal error conditions resulting from several sources. Corrupted messages are flagged by any node detecting an error. Such messages are aborted and will be retransmitted automatically.

The protocol requires that receivers **acknowledge** reception of the message by changing the content of the ACK FIELD. The ACK FIELD is two bits long and contains the ACK SLOT and the ACK DELIMITER. In the ACK FIELD the transmitting station sends two recessive bits. All receiver nodes that detect a correct message (after CRC checks), inform the sender by changing the recessive bit of the ACK SLOT into a dominant bit.



CAN - CRC checks

The CRC portion of the frame is obtained by selecting the input polynomial (to be divided) as the stream of bits from the START OF FRAME bit (included) to the DATA FIELD (if present) followed by 15 zeros.

This polynomial is divided by the generator:

$$X^{15} + X^{14} + X^{10} + X^8 + X^7 + X^4 + X^3 + 1.$$

The remainder of the division is the CRC SEQUENCE portion of the frame



CAN - Error types

There are 5 different error types:

- **BIT ERROR:** a unit that is sending a bit on the bus also monitors the bus. A BIT ERROR has to be detected at that bit time, when the bit value that is monitored is different from the bit value that is sent. Exceptions are the recessive bits sent as part of the arbitration process or the ACK SLOT.
- **STUFF ERROR:** a STUFF ERROR is detected at the 6th consecutive occurrence of the same bit in a message field that is subject to stuffing.
- **CRC ERROR:** if the CRC computed by the receiver differs from the one stored in the message frame.
- **FORM ERROR:** when a fixed-form bit field contains one or more illegal bits. (Note, that for a Receiver a dominant bit during the last bit of END OR FRAME is not treated as FORM ERROR, this is the possible cause of inconsistent message omission and duplicates, as further explained in the chapter on reliability).
- **ACKNOWLEDGMENT ERROR:** detected by a transmitter if a recessive bit is found on the ACK SLOT.



CAN - Common CAN type

The main CAN bus that are used on the vehicle are classified on the basis of the baud rate.

The most used network are:

- C-CAN: the baud rate is 500 kb/s;
- B-CAN: the baud rate is 50 kb/s;
- BH-CAN: the baud rate is 125 kb/s;
- FD-CAN: the baud rate is 1Mb/s.

Usually on the C-CAN there are for sure all the ECUs that are emission relevant in accord to EOBD normative.

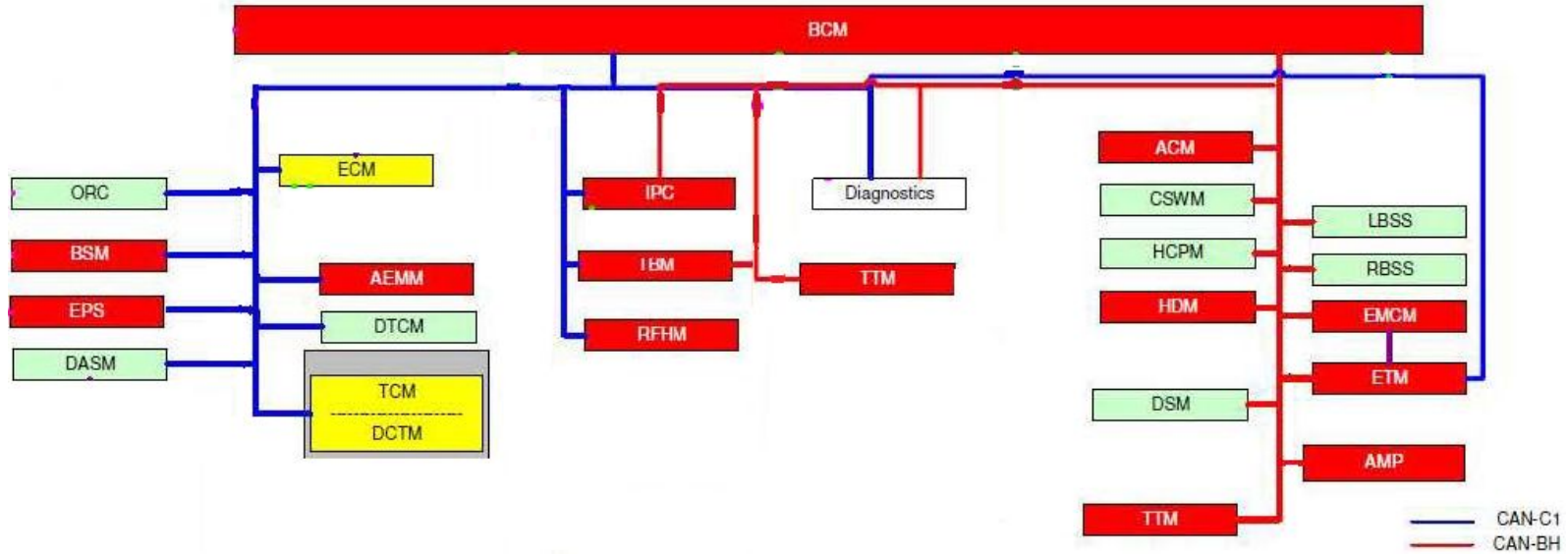
Usually on the BH-CAN there are all the ECUs that manage multimedial information

Bit rate	bit time	bus length
1 Mb/s	1 μ s	25m
800 kb/s	1.25 μ s	50m
500 kb/s	2 μ s	100m
250 kb/s	4 μ s	250m
125 kb/s	8 μ s	500m
62.5 kb/s	16 μ s	1000m
20 kb/s	50 μ s	2500m
10 kb/s	100 μ s	5000m



CAN - Main vehicle ECU

In the figure below there are an example of CAN networks for a vehicle with the main ECUs.





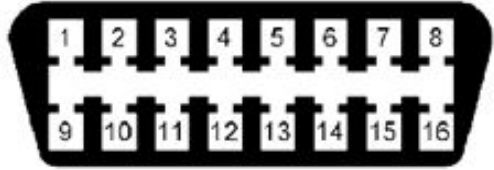
CAN - Access to CAN bus

It's possible to access on the CAN bus by the diagnostic socket also known as EOBD connector.

This connector usually is fitted on the driver side of the vehicle between driver door and steering wheel.

The features of this connector are:

- it has 16 pin;
- the pin 6 and 14 are reserved for EOBD CAN;
- the pin 4 and 5 are reserved for ground;
- the pin 16 is reserved for Power supply (+12V);
- the other pin are configurable from vendor.



PIN	DESCRIPTION	PIN	DESCRIPTION
1	Vendor Option	9	Vendor Option
2	J1850 Bus +	10	J1850 Bus -
3	Vendor Option	11	Vendor Option
4	Chassis Ground	12	Vendor Option
5	Signal Ground	13	Vendor Option
6	CAN (J-2234) High	14	CAN (J-2234) Low
7	ISO 9141-2 K-Line	15	ISO 9141-2 L-Line
8	Vendor Option	16	Battery Power

OBD-II Connector and Pinout



CANalyzer



CANalyzer - Introduction

CANalyzer is a powerful tool with intuitive operation for analysis and stimulation of bus communication. Use CANalyzer to check whether and what type of communication is occurring on the bus. It can also be used to send or log data

CANalyzer Options for:

K-Line **FlexRay** MOST
Ethernet WLAN **Car2x**
Aerospace **LIN** CANopen
J1939 J1587 / J1708
CAN/CAN FD Scope

Application Areas

Analysis

Diagnostics

Logging

Stimulation

CAN

LIN

FlexRay

MOST

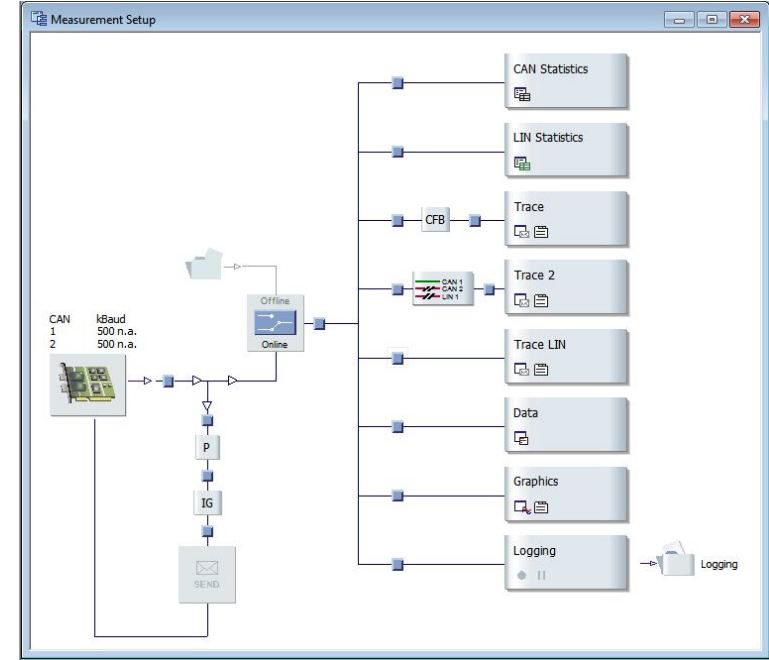
Bus system



Configuration - Measurement setup window

The data flow is graphically represented and configured in the Measurement Setup.

- **Define data source** (online/offline) The simulated bus or the real bus connected via the hardware (e.g. CANcardXL) serves as the online data source. A file with logged data serves as the offline data source.
- **Insert analysis windows** Data can be shown differently in the individual windows depending on analysis requirements
- **Insert CAPL program nodes** A CAPL program node can be used for such tasks as filtering data or implementing various arithmetic operations.
- **Insert filters** Filters can be used to obtain a more understandable representation of the data; they define which data should be passed and which should be explicitly blocked.
- **Log data** For an analysis after the measurement, data can be logged in a logging file that can later be reused as an offline data source and replayed.

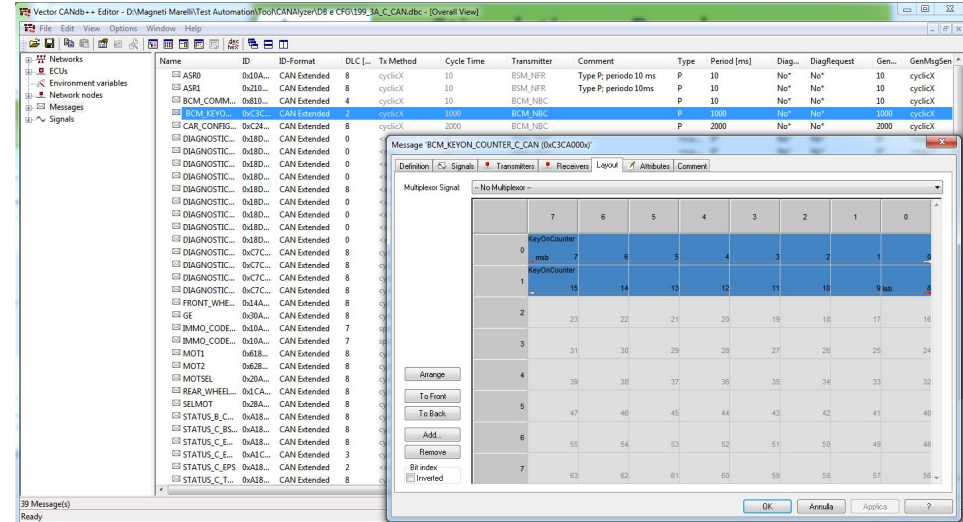




Configuration - Vector CANdb++

The **CANdb++ Editor** offers options for visualizing the data contained in the DBC databases and for modifying the data easily. For this purpose it contains specialized, application orientated views on the data:

- Creation of DBC files
- Adding messages and signals to existing DBC databases
- Definition of transmission and reception relations
- Defining environment variables for a CANoe simulation

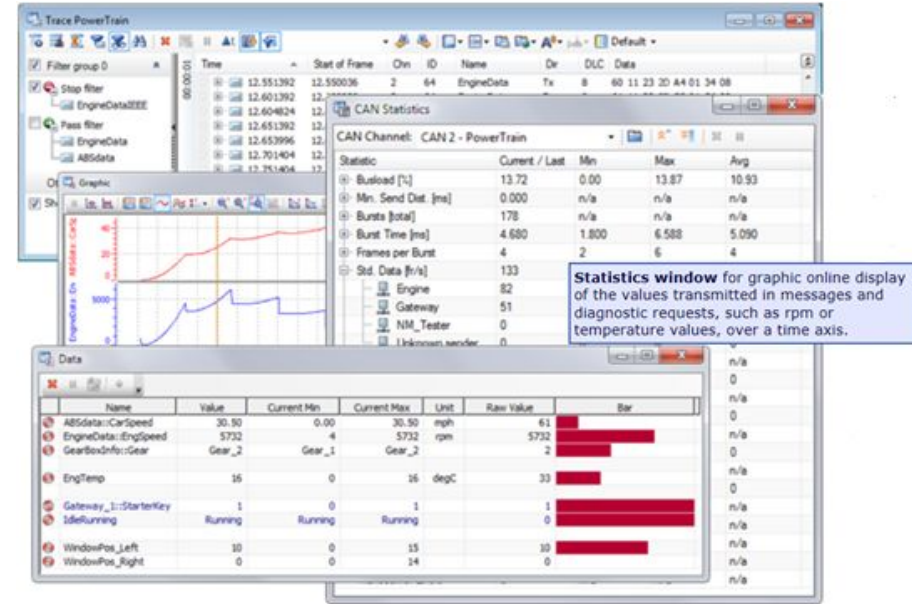




CANalyzer - Analysis

As a CANalyzer user you can analyze the bus communication of ECUs or entire systems at your work desk or right in the vehicle. These intelligent analysis windows support you. Other CANalyzer windows and blocks:

- **Measurement setup** for graphic representation and parameterization of function blocks and evaluation functions
- **Interactive Generator** for stimulating the buses and for easy sending of modified signals
- **Logging/Replay** for logging and later analysis or replay of measurements
- **Triggers and filters** to make specific reactions to bus events and to reduce the volume of displayed or logged data
- **Write window** for system messages and user-specific outputs from CAPL programs

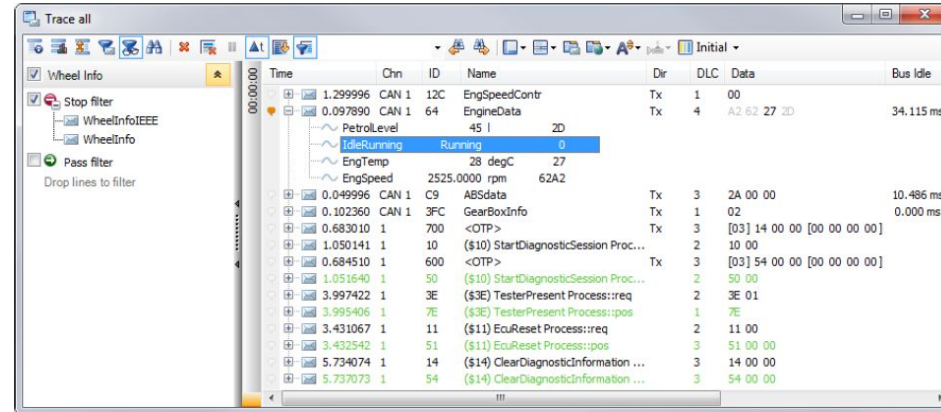




CANalyzer - Trace window

Bus activities – such as the sending of messages or Error Frames – are listed in the **Trace Window**. Individual signal values may be displayed for each message.

- **Insert filters** There are various types of filters in the Trace Window. They can be used to reduce the amount of data displayed, and data can even be deleted from the data stream.
- **Hide unchanged data** To improve ease of viewing, data that does not change is slowly faded or removed entirely from the screen.
- **Color events** Important events and messages can be highlighted in color.
- **Set markers** Markers can be set to identify and quickly find events. The marker is assigned to an event and therefore to its time stamp as well. The set markers can also be displayed in other analysis windows.
- **Show statistics** Various aspects of messages/signals, including their values, can be displayed in different views in detail. Differences between the time stamps or signal values may also be calculated.
- **Log data** It is possible to export some or all of the Trace Window contents. Files that have already been exported can be converted to a different format afterwards, e.g. to further process the same dataset in different programs.

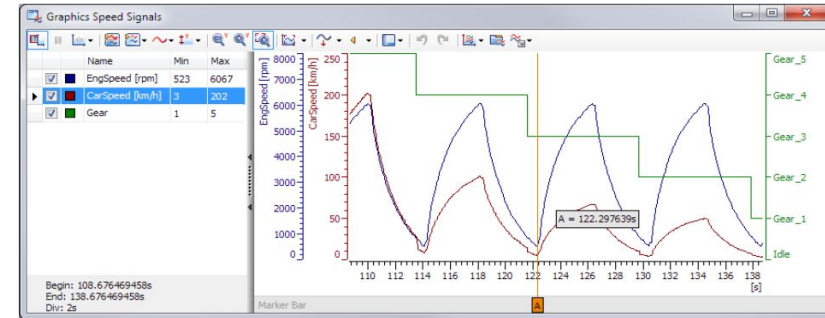




CANalyzer - Graphics window

The Graphics Window is used to graphically display the values of signals, environment data and diagnostic parameters as curves.

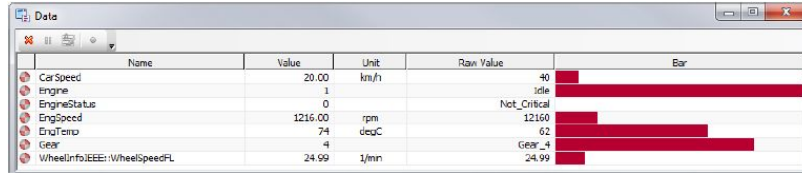
- **Show measurement markers/difference markers** Measurement or difference markers can be used to perform absolute or relative analyses of measurement values. The measurement marker can be synchronized to the Trace Window display.
- **Set markers** Markers can be set to identify and quickly find events. The marker is assigned to one event and therefore to its time stamp as well. The set markers can also be displayed in other analysis windows.
- **Show measurement columns** In the legend, global or local minima and maxima may be shown for each signal, or Y-differences between signals of the same type can be displayed.
- **Show statistics** Statistical data such as minimum, maximum, mean value and standard deviation can be compiled for selected signals or all signals of the Graphics Window.
- **Log data** Signals of the Graphics Windows can be logged automatically or manually during the measurement. This involves extracting the signals from the messages and saving them in binary form in signal based MDF files. In the Graphics Window, the entire signal waveform or just a visible section of the signal waveform can be saved to a file.





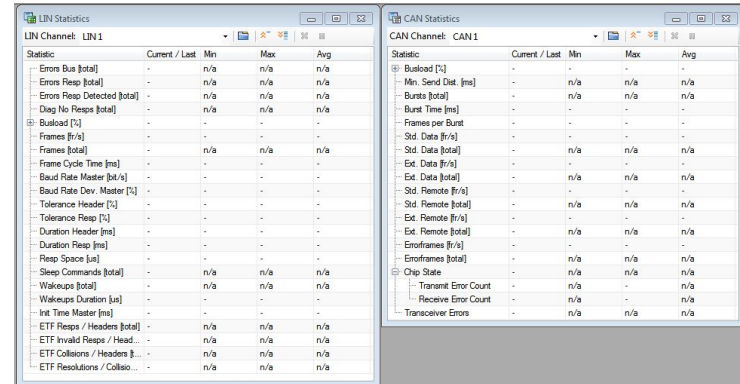
CANalyzer - Statistics and data windows

The Data Window is used to display the values of signals, system variables and diagnostic parameters in different types of representation.



Name	Value	Unit	Raw Value	Bar
CarSpeed	20.00	km/h	40	
Engine	1		Idle	
EngineStatus	0		Not_Critical	
EngSpeed	1216.00	rpm	12160	
EngTemp	74	degC	62	
Gear	4		GEAR_4	
WheelInfo[IEEE1576]WheelSpeedFL	24.99	1/min	24.99	

The Statistics Window shows statistical information about bus activities (CAN, LIN, FlexRay) during a measurement. This includes such information as bus load on node and frame level, burst counter/duration, counters/rates for frames and errors, and controller states.



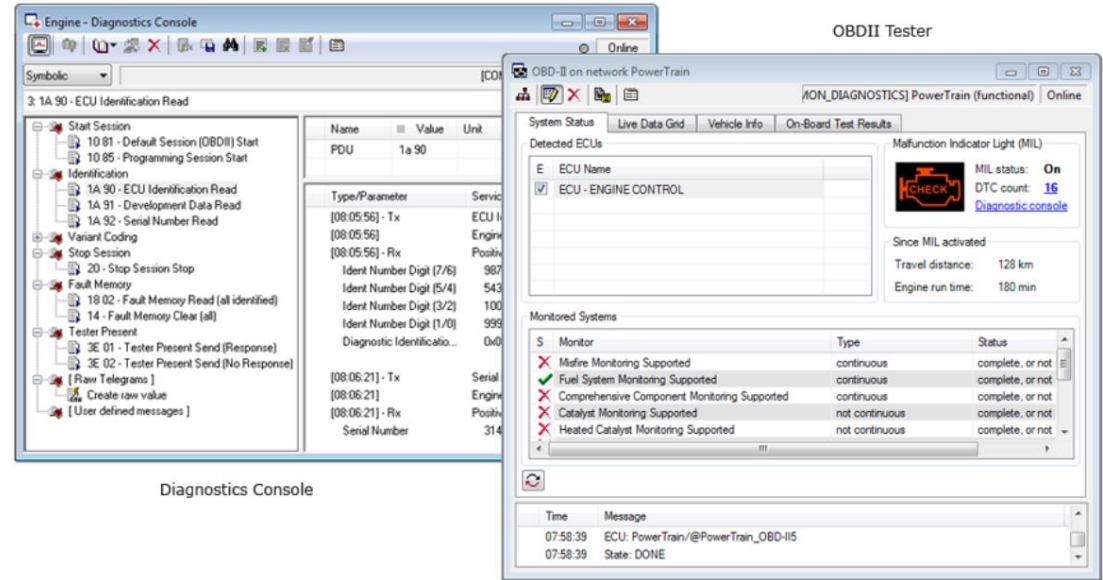
Statistic	Current / Last	Min	Max	Avg
Errors Bus [total]	-	n/a	n/a	n/a
Errors Resp [total]	-	n/a	n/a	n/a
Errors Resp Detected [total]	-	n/a	n/a	n/a
Diag No Resp [total]	-	n/a	n/a	n/a
Busload [%]	-	-	-	-
Frames [r/s]	-	-	-	-
Frames [total]	-	n/a	n/a	n/a
Frame Cycle Time [ms]	-	-	-	-
Baud Rate Dev. Master [%]	-	-	-	-
Tolerance Header [%]	-	-	-	-
Tolerance Resp [%]	-	-	-	-
Duration Header [ms]	-	-	-	-
Duration Resp [ms]	-	-	-	-
Resp Space [us]	-	-	-	-
Sleep Commands [total]	-	n/a	n/a	n/a
Wakeup [total]	-	n/a	n/a	n/a
Wakeup Duration [us]	-	-	-	-
Int Time Master [ms]	-	-	-	-
ETP Resps / Headers [total]	-	n/a	n/a	n/a
ETP Invalid Resps / Head...	-	n/a	n/a	n/a
ETP Collisions / Headers...	-	n/a	n/a	n/a
ETP Resolutions / Collis...	-	n/a	n/a	n/a

Statistic	Current / Last	Min	Max	Avg
Busload [%]	-	-	-	-
Min Send Dist. [ms]	-	n/a	n/a	n/a
Bursts [total]	-	n/a	n/a	n/a
Burst Time [ms]	-	-	-	-
Frames per Burst	-	-	-	-
Std. Data [r/s]	-	-	-	-
Std. Data [total]	-	n/a	n/a	n/a
Ext. Data [r/s]	-	-	-	-
Ext. Data [total]	-	n/a	n/a	n/a
Std. Remote [r/s]	-	-	-	-
Std. Remote [total]	-	n/a	n/a	n/a
Ext. Remote [r/s]	-	-	-	-
Ext. Remote [total]	-	n/a	n/a	n/a
Errframes [r/s]	-	n/a	n/a	n/a
Errframes [total]	-	n/a	n/a	n/a
Chip State	-	n/a	n/a	n/a
Transmit Error Count	-	n/a	-	n/a
Receive Error Count	-	n/a	-	n/a
Transceiver Errors	-	n/a	n/a	n/a



CANalyzer - Diagnostic section 1\2

Using the Diagnostic Feature Set contained in CANalyzer, you can analyze diagnostic communication in accordance with the KWP2000 or UDS standard. Here, CANalyzer can be used as the diagnostic tester for ECU diagnostics; parameterization is via ODX or CANDelaStudio description files. In addition, a complete OBD-II Tester is already integrated in CANalyzer.

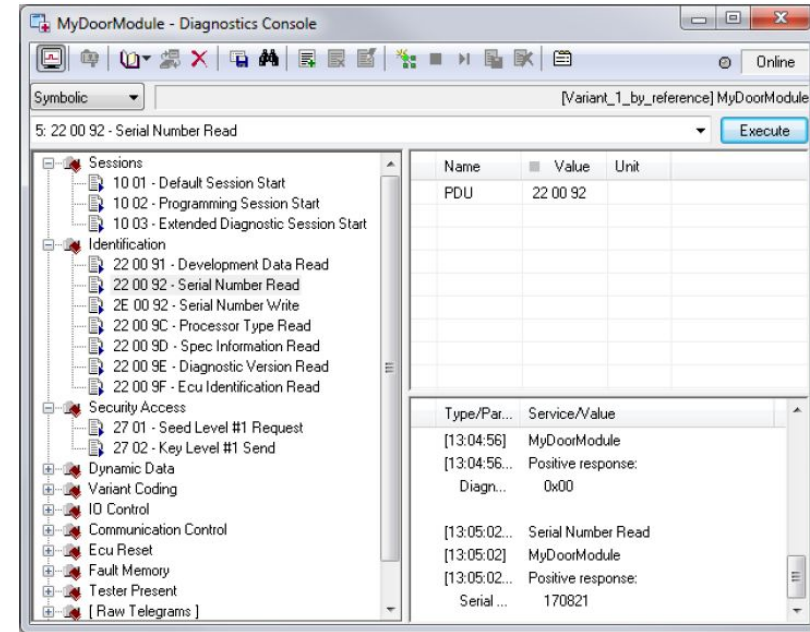




CANalyzer - Diagnostic section 2\2

CANalyzer is used essentially to define and send diagnostic services for KWP2000 or UDS.

- Support of all important diagnostic description formats for KWP2000 and UDS (ISO 14229):
 - ODX 2.0.1/2.2.0 as PDX files
 - CANdelaStudio (CDD)
- Option of modifying key diagnostic communication parameters of the diagnostic description (transport and diagnostic layer) in the Diagnostic/ISO-TP Configuration dialog
- Basic Diagnostic Editor for quickly defining simple diagnostic services, if no diagnostic description is available
- Interactive diagnostic tester with Diagnostic Console, Fault Memory Window and Diagnostic Session Control with configurable Security-DLL
- Preconfigured OBD-II tester with related Diagnostic Console and Fault Memory Window
- Support of several addressing schemes (e.g. normal, extended, normal fixed and mixed) and addressing types (functional/physical)
- Logging and replay of diagnostic sequences via macros



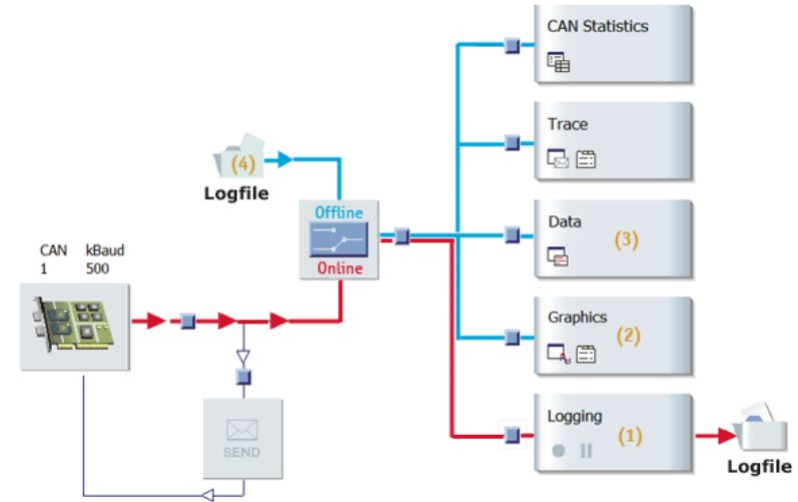


CANalyzer - Logging

Use CANalyzer to log data and replay them for post-measurement analysis. The import/export functions allow time-independent processing of the logged bus communication.

Logging workflow

- Logging is performed via special logging blocks in the measurement setup (1). You can also log directly from the Graphic window (2) and Data window (3).
- The logged data can be played back into the system for offline analysis (4).





CANalyzer - Stimulation

In addition to observing and analyzing the data traffic of ECUs, you can also use CANalyzer to influence the data traffic. To do this, you can simply send messages from preconfigured user interfaces or define signal values in CANalyzer and send the related messages.

Configurable user interfaces

- Predefined by CANalyzer such as the Interactive Generator
- Can be user created (via "Panels")

Programs

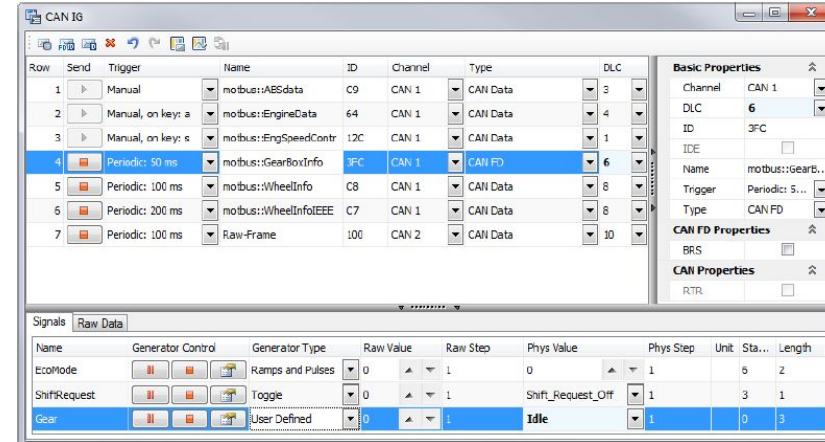
- Predefined programming steps make it easy to create command sequences in the **Visual Sequencer**
- Independent programming via CAPL or .NET



Stimulation - Interactive Generator Block

The Interactive Generator (IG) can be used to send messages as well as to set the corresponding signal values. This is an easy way to stimulate the bus:

- **Define messages** Messages can be defined manually in the send list or with a database. The properties of the messages can be customized.
- **Send messages** Messages which are configured in the send list can be sent periodically when a specific screen button is pressed or by pressing a predefined keyboard key.
- **Change signal values** In the Interactive Generator, the raw data of each message can be modified in the signal list. For configured signals of a specific message signal waveforms (signal curves) may be defined with the **integrated Signal Generator**. Raw data and signal values can be easily sent with the associated message.



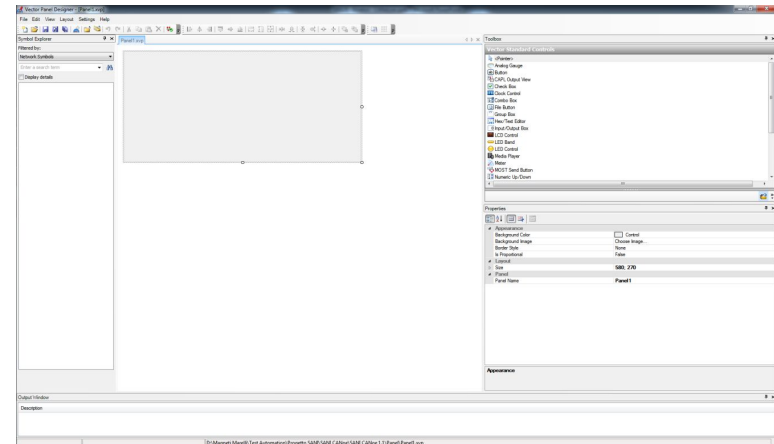


Stimulation - Panels

Panels are graphical elements that can be used to modify signal and variable values and display them with controls such as sliders or pointer instruments. Panels are used to display the analysis data from CAPL programs, for example, or to control CAPL programs and transfer values into the programs.



The **Panel Designer** can be used to conveniently create such panels. For example, it is easy to link a symbol to a control by drag & drop. The individual panels and controls are configured via the constantly open **Properties Window**, and a whole series of useful alignment functions ensure an optimal layout.





Stimulation - CAPL 1\2

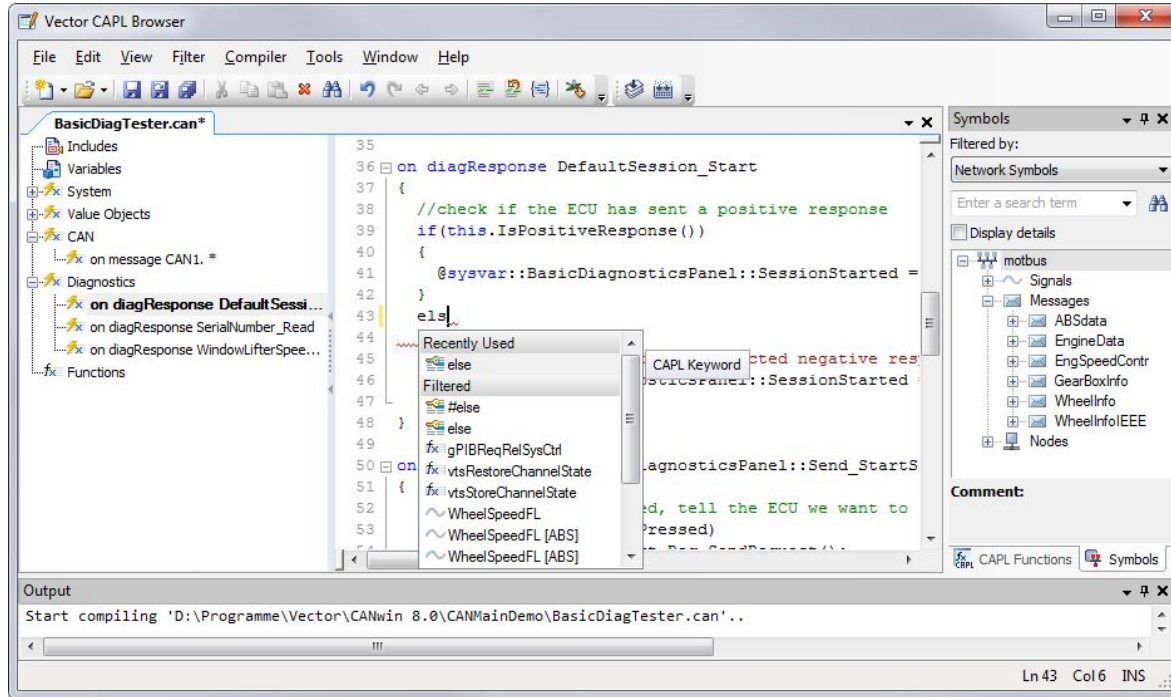
The CAPL (Communication Access Programming Language) programming language extends the functional scope of CANalyzer tremendously:

- **C-Like Syntax** Can be learned quickly since it is based on the C programming language
- **Event-Oriented Control** CAPL is an event-controlled programming language. In contrast to C, special predefined event handlers (event procedures) are available in CAPL, which are always executed whenever a specific event occurs (if time controlled then triggered by the hardware or internal to CANalyzer).
- **Symbolic Access** Supports symbolic access to all database information such as messages and signals. Signal values can be used directly in their physical form.
- **Application-Specific Language Extensions** The language has been extended with special functions for quick implementation of problem solutions in various use scenarios
 - **Analysis** CAPL can be used in the analysis of measurement results – either online and offline. One simple task might be to count the occurrences of a specific event or perform computations with the contents of certain signals.
 - **Stimulation** You can also use CAPL to generate messages to stimulate ECUs. This relieves the developer of routine work tasks. Signals and messages of the bus are defined in the database (e.g. in DBC or LDF).
- **Flexible extension** by external libraries



Stimulation - CAPL 2\2

The functionality of the CAPL Browser goes beyond that of an editor for CAPL programs.



CONTACTS

Bruno D'Angelo

bruno.Dangelo@kineton.it

Via G. Porzio,
Centro Direzionale Isola B3
80143 Napoli Via G. Porzio

www.kineton.it

